

Homework 3

Due : April 12th at 11 : 59 pm

Deliverables. Submit a single PDF of your write-up to Gradescope *HW3 Write-Up*.

Start each problem on a new page.

Honor Code

Write and sign the following statement:

"I certify that all solutions in this document are entirely my own and that I have not looked at anyone else's solution. I have given credit to all external sources I consulted."

"I certify that all solutions in this document are entirely my own and that I have not looked at anyone else's solution. I have given credit to all external sources I consulted."

X James Dink

1 Newton Might Have Been a Logistics Expert

Given examples $x_1, x_2, \dots, x_n \in \mathbb{R}^d$ and associated labels $y_1, y_2, \dots, y_n \in \{0, 1\}$, the cost function for *unregularized* logistic regression is

$$J(w) = - \sum_{i=1}^n \left(y_i \log s_i + (1 - y_i) \log(1 - s_i) \right) \quad \log \left(\frac{1}{1 + e^{-x_i}} \right)$$

where $s_i = s(x_i \cdot w)$, $w \in \mathbb{R}^d$ is a weight vector, and $s(\gamma) = 1/(1 + e^{-\gamma})$ is the logistic function.

Define the $n \times d$ design matrix X (whose i^{th} row is $x_i^\top$), the label n -vector $y = [y_1 \dots y_n]^\top$, and $s = [s_1 \dots s_n]^\top$. For an n -vector a , let $\log a = [\log a_1 \dots \log a_n]^\top$. The cost function can be rewritten in vector form as

$$J(w) = -y \cdot \log s - (\mathbf{1} - y) \cdot \log (\mathbf{1} - s).$$

Further, recall that for a real symmetric matrix $A \in \mathbb{R}^{d \times d}$, there exist U and Λ such that $A = U\Lambda U^\top$ is the eigendecomposition of A . Here Λ is a diagonal matrix with entries $\{\lambda_1, \dots, \lambda_d\}$. An alternative notation is $\Lambda = \text{diag}(\lambda_i)$, where $\text{diag}()$ takes as input the list of diagonal entries, and constructs the corresponding diagonal matrix. This notation is widely used in libraries like **numpy**, and is useful for simplifying some of the expressions when written in matrix-vector form. For example, we can write $\mathbf{s} = \text{diag}(s_i) \mathbf{1}$.

Hint: Recall matrix calculus identities. The elements in **bold** indicate vectors for clarity (although are not bolded in the problem statement). In the first identity below, α denotes a *scalar*-valued function of $\mathbf{x}$ (not a vector).

$$\begin{aligned} \nabla_{\mathbf{x}} \alpha \mathbf{y} &= (\nabla_{\mathbf{x}} \alpha) \mathbf{y}^\top + \alpha \nabla_{\mathbf{x}} \mathbf{y} & \nabla_{\mathbf{x}} (\mathbf{y} \cdot \mathbf{z}) &= (\nabla_{\mathbf{x}} \mathbf{y}) \mathbf{z} + (\nabla_{\mathbf{x}} \mathbf{z}) \mathbf{y}; \\ \nabla_{\mathbf{x}} \mathbf{f}(\mathbf{y}) &= (\nabla_{\mathbf{x}} \mathbf{y}) (\nabla_{\mathbf{y}} \mathbf{f}(\mathbf{y})); & \nabla_{\mathbf{x}} g(\mathbf{y}) &= (\nabla_{\mathbf{x}} \mathbf{y}) (\nabla_{\mathbf{y}} g(\mathbf{y})); \end{aligned}$$

and $\nabla_{\mathbf{x}} C \mathbf{y}(\mathbf{x}) = (\nabla_{\mathbf{x}} \mathbf{y}(\mathbf{x})) C^\top$, where C is a constant matrix.

- (7 pts) Derive the gradient $\nabla_w J(w)$ of cost $J(w)$ as a matrix-vector expression. Also derive *all intermediate derivatives* in matrix-vector form. Do NOT specify them (**including the intermediates**) in terms of their individual components (e.g. w_i for vector w).
- (1 pt) Derive the Hessian $\nabla_w^2 J(w)$ for the cost function $J(w)$ as a matrix-vector expression.
- (2 pts) Write the matrix-vector update law for one iteration of Newton's method, substituting the gradient and Hessian of $J(w)$.

$$J(w) = (-y \log s - (1-y) \log(1-s))$$

$$a) \nabla_w J(w) = \frac{\partial J}{\partial s} \cdot \frac{\partial s}{\partial w}$$

$$\hookrightarrow \nabla_s J(w) = -\frac{y}{s} + \frac{1-y}{1-s} = \frac{-y(1-s) + s(1-y)}{s(1-s)} = \frac{s-y}{s(1-s)}$$

$$\nabla_w s = \nabla_w(x_w) \cdot \nabla_{x_w}(s(x_w)) = X^T \cdot \text{diag}(s(1-s))$$

chain rule

$$= \frac{s-y}{s(1-s)} X^T \text{diag}(s(1-s)) = X^T (s-y)$$

$$\hookrightarrow \text{diag}(s \odot (1-s)) = s \odot (1-s)$$

$$\text{diag}(v) \cdot u = v \odot u$$

$$b) \nabla_w^2 J(w) = \frac{\partial s}{\partial w} \cdot \frac{\partial}{\partial s} (X^T (s-y)) = \nabla_w(x_w) \nabla_{x_w}^s(x_w) X = X^T s \odot (1-s) X = \boxed{X^T D X}$$

$D = \text{diag}(s \odot (1-s))$

$$c) \text{Newton's update method: } w^{(t+1)} = w^{(t)} - (\nabla_w^2 J)^{-1} \nabla_w J$$

$$\boxed{w^{(t+1)} = w^{(t)} - (X^T D X)^{-1} X^T (s-y)}$$

$$\Rightarrow s(x_w) = \frac{1}{1 + e^{-x_w}}$$

$$\frac{\partial s_i}{\partial x_{w_i}} = \frac{e^{-x_{w_i}}}{(1 + e^{-x_{w_i}})^2} = \frac{e^{-x_{w_i}}}{(1 + e^{-x_{w_i}})^2}$$

$$= \frac{1}{1 + e^{-x_{w_i}}} \cdot \frac{(1 + e^{-x_{w_i}}) - 1}{(1 + e^{-x_{w_i}})^2} = s_i(1-s_i)$$

$$s'_i(x_w)_j = \begin{cases} s_i(1-s_i) & \text{if } i=j \\ 0 & \text{otherwise} \end{cases}$$

2 A Coordinated Leap of Faith

In this question we will study a special case of stochastic gradient descent (SGD), namely *coordinate descent*.

Recall the setting of SGD. We have a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ which we want to minimize. However, instead of taking gradient descent steps on f , we take random directions based on a gradient oracle g . Specifically, given a $g(w; \xi)$ and a probability distribution p over ξ such that for every fixed $w \in \mathbb{R}^d$ we have $\mathbb{E}_\xi[g(w; \xi)] = \nabla f(w)$, SGD constructs the sequence:

$$w_{t+1} = w_t - \alpha_t g(w_t; \xi_t),$$

where $\{\xi_t\}_{t \geq 0}$ is an i.i.d. sequence from the distribution p .

In coordinate descent, we assume a very particular form for the gradient oracle. In particular, we assume that $\xi \in \{1, \dots, d\}$, that $p = \text{Unif}(\{1, \dots, d\})$ and that

$$g(w; \xi) = d \langle \nabla f(w), e_i \rangle e_i, \quad (1)$$

where $e_i \in \mathbb{R}^d$ is the i -th standard basis vector in $\mathbb{R}^d$, and $\langle \cdot, \cdot \rangle$ denotes the standard Euclidean inner product throughout. Coordinate descent can be computationally beneficial when it is possible to evaluate the i -th element of a gradient $\nabla f(w)$ much quicker than evaluating the full gradient vector.

- (a) (2 pts) Show that stochastic gradient proposed in Equation 1 is a valid stochastic gradient, i.e. $\mathbb{E}_\xi[g(w; \xi)] = \nabla f(w)$ for every fixed $w \in \mathbb{R}^d$.
- (b) (4 pts) Suppose that f is twice differentiable. For every $i = 1, \dots, d$, define

$$L_i = \sup_{w \in \mathbb{R}^d} (\nabla^2 f(w))_{ii}$$

Show that for every $x, y \in \mathbb{R}^d$ satisfying $\text{supp}(x - y) \subseteq \{i\}$, the following quadratic upper bound is valid:

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L_i}{2} \|y - x\|_2^2.$$

Note: $\sup$ refers to the *supremum*. Unlike a maximum, the supremum need not be attained at any particular w ; it simply bounds all possible values of the i -th diagonal entry of the Hessian.

Additionally, supp refers to the *support* of a function. The support of a vector-valued function refers to the set of coordinate indices where its components are not zero. In this case, $\text{supp}(x - y) \subseteq \{i\}$ means x, y such that their difference vector has non-zero entries only at the i -th index.

- (c) (4 pts) First, we define

$$L_{\max} = \max_{1 \leq i \leq d} L_i$$

Use the previous part to show that if coordinate descent is run with a constant step size $\alpha_t = \alpha = \frac{1}{L_{\max} d}$, then

$$\mathbb{E}[f(w_{t+1})] \leq \mathbb{E} \left[f(w_t) - \frac{1}{2dL_{\max}} \|\nabla f(w_t)\|_2^2 \right].$$

Conclude furthermore that if for some $m > 0$ we have that $\frac{1}{2}\|\nabla f(w)\|_2^2 \geq m(f(w) - f_\star)$ for every $w \in \mathbb{R}^d$, then:

$$\mathbb{E}[f(w_t) - f_\star] \leq \left(1 - \frac{m}{dL_{\max}}\right)^t \mathbb{E}[f(w_0) - f_\star].$$

Hint: When taking the expectation over the random coordinate index $i_t \sim \text{Unif}(\{1, \dots, d\})$, you may use the identity $\mathbb{E}[e_{i_t} e_{i_t}^\top] = \frac{1}{d} I_d$ together with the tower property of expectation.

- (d) (2 pts) Suppose now and for the remainder of the parts that $f(w) = \frac{1}{2}w^\top A w - w^\top b$, where A is positive definite. What does the coordinate descent update look like in terms of the rows of A and the entries of b ? What are $L_{\max}$ and m in terms of A ?
- (e) (3 pts) Compare the computational complexity of one iteration of coordinate descent versus one iteration of (full) gradient descent.
- (f) (3 pts) Note that gradient descent with $\alpha_t = 1/L$ satisfies $f(w_t) - f_\star \leq (1 - m/L)^t (f(w_0) - f_\star)$, where $L = \lambda_{\max}(A)$. First, prove that $L_{\max} \leq L$. Then, compare the overall complexity for both coordinate descent and gradient descent to reach a ε suboptimal solution.

Hint: Recall the variational characterization of the maximum eigenvalue of a symmetric matrix: $\lambda_{\max}(A) = \max_{\|v\|_2=1} v^\top A v$.

$$a) E_{\xi}[g(w; \xi)] = \sum_{i=1}^d \frac{1}{d} \cdot g(w; i) = \sum_{i=1}^d \frac{1}{d} \cdot \langle \nabla f(w), e_i \rangle e_i = \sum_{i=1}^d [\nabla f(w)]_i \cdot e_i = \nabla f(w)$$

Uniform $\{1, \dots, d\}$
extracts the i -th component of $\nabla f(w)$
same as $v = \sum_{i=1}^d v_i \cdot e_i$

b) $\text{supp}(x-y) \subseteq \{i\}$ $y = x + te_i$ for some scalar $t = (y-x)_i$ and $\|y-x\|_2^2 = t^2$
 this means x and y differ only in the i -th coordinate

Apply Taylor's theorem

$$f(y) = f(x) + \langle \nabla f(x), y-x \rangle + \int_0^1 (1-s)(y-x)^T \nabla^2 f(x+s(y-x))(y-x) ds$$

Using $y-x = te_i$:

$$(y-x)^T \nabla^2 f(x+s(y-x))(y-x) = t^2 \cdot e_i^T \nabla^2 f(x+s(y-x)) e_i = t^2 \cdot (\nabla^2 f(x+s(y-x)))_{ii}$$

$e_i^T A e_i$ chooses $A(i,i)$

By definition, $(\nabla^2 f(w))_{ii} \leq L$ for all $w \in \mathbb{R}^d$

$$\int_0^1 (1-s) t^2 \cdot (\nabla^2 f(\cdot))_{ii} ds \leq t^2 L; \int_0^1 (1-s) ds = \frac{1}{2} \quad \left] \text{bound by } L; \right.$$

$$f(y) \leq f(x) + \langle \nabla f(x), y-x \rangle + \frac{L}{2} t^2 = f(x) + \langle \nabla f(x), y-x \rangle + \frac{L}{2} \|y-x\|_2^2$$

c) coordinate descent update is

$$w_{t+1} = w_t - \alpha g(w_t, \xi_t) = w_t - \alpha d \langle \nabla f(w_t), e_i \rangle e_i$$

w_{t+1} and w_t differ only in coordinate i , so $\text{supp}(w_{t+1} - w_t) \subseteq \{i\}$

$$f(w_{t+1}) \leq f(w_t) + \langle \nabla f(w_t), w_{t+1} - w_t \rangle + \frac{L}{2} \|w_{t+1} - w_t\|_2^2$$

With $w_{t+1} - w_t = -\alpha d \langle \nabla f(w_t), e_i \rangle e_i$,

$$f(w_{t+1}) \leq f(w_t) - \alpha d \langle \nabla f(w_t), e_i \rangle^2 + \frac{L}{2} \alpha^2 d^2 \langle \nabla f(w_t), e_i \rangle^2$$

Using tower property, taking expectation over i , let $\alpha = \frac{1}{L_{\max} d}$:

$$E_i[f(w_{t+1})] \leq f(w_t) - \alpha d \cdot E_i[\langle \nabla f(w_t), e_i \rangle^2] + \frac{L_{\max}}{2} \alpha^2 d^2 \cdot E_i[\langle \nabla f(w_t), e_i \rangle^2]$$

$$\hookrightarrow E_i[\langle \nabla f(w_t), e_i \rangle^2] = E_i[\nabla f(w_t)^T e_i e_i^T \nabla f(w_t)] = \nabla f(w_t)^T \cdot \frac{1}{d} I_d \cdot \nabla f(w_t) = \frac{1}{d} \|\nabla f(w_t)\|_2^2$$

$$\langle u, v \rangle^2 = u^T v v^T u$$

$$E[f(w_{t+1})] \leq f(w_t) + \frac{1}{L_{\max} d} \cdot \frac{1}{d} \|\nabla f(w_t)\|_2^2 + \frac{L_{\max}}{2} \cdot \frac{1}{L_{\max}^2 d^2} \cdot d^2 \cdot \frac{1}{d} \|\nabla f(w_t)\|_2^2$$

$$= f(w_t) - \frac{1}{L_{\max} d} \|\nabla f(w_t)\|_2^2 + \frac{1}{2 L_{\max} d} \|\nabla f(w_t)\|_2^2 = f(w_t) - \frac{1}{2 d L_{\max}} \|\nabla f(w_t)\|_2^2$$

Taking full expectation w/o tower property,

$$E[f(w_{t+1})] \leq E\left[f(w_t) - \frac{1}{2 d L_{\max}} \|\nabla f(w_t)\|_2^2\right]$$

Subtract f_* from both sides and apply condition $\frac{1}{2} \|\nabla f(w)\|_2^2 \geq m(f(w) - f_*)$,

$$E[f(w_{t+1}) - f_*] \leq E[f(w_t) - f_*] - \frac{1}{d L_{\max}} E\left[\frac{1}{2} \|\nabla f(w_t)\|_2^2\right] \quad \begin{array}{l} \text{substitute out} \\ \text{of bound} \end{array}$$

$$\leq E[f(w_t) - f_*] - \frac{m}{d L_{\max}} E[f(w_t) - f_*]$$

$$= \left(1 - \frac{m}{dL_{\max}}\right) E[f(u_t) - f_*]$$

$$E[f(w_{t+1}) - f_*] \leq \underbrace{\left(1 - \frac{m}{dL_{\max}}\right)^t}_{\text{applied recursively } t \text{ times}} E[f(u_t) - f_*]$$

d) Suppose $f(w) = \frac{1}{2} w^T A w - w^T b$, A is pos def

$$\nabla f(w) = A w - b$$

$$w_{t+1} = w_t - \alpha \cdot \langle \nabla f(w_t), e_i \rangle e_i = w_t - \underbrace{\frac{1}{L_{\max}} d \cdot (A w_t - b)_i}_{\text{update only changes coordinate } i} e_i \Rightarrow w_{t+1} = w_t - \frac{1}{L_{\max}} (A w_t - b)_i e_i$$

$$\hookrightarrow (w_{t+1})_i = (w_t)_i - \frac{1}{L_{\max}} (A w_t - b)_i e_i$$

$\frac{1}{2} \|\nabla f(w)\|_2^2 \geq m(f(w) - f_*)$ is satisfied by

$$\nabla^2 f(w) = A \text{ (constant)}$$

$$m = \lambda_{\min}(A) \text{ (smallest eigenvalue of } A)$$

$$\hookrightarrow (\nabla^2 f(w))_{ii} = A_{ii} \Rightarrow L_i = A_{ii} \Rightarrow L_{\max} = \max_i A_{ii}$$

e) coordinate descent (one step): $(A w_t)_i$ is a dot product of one row A with w_t , which is $O(d)$ operations.

full gradient descent (one step): $A w_t - b$ is the full matrix-vector product, which is $O(d^2)$ operations.

coordinate descent is d times cheaper than full gradient descent.

f) Prove $L_{\max} \leq L$:

$$L_{\max} = \max_i A_{ii} \quad (\text{part d})$$

$$A_{ii} = e_i^T A e_i \leq \underbrace{\max_{\|v\|_2=1} v^T A v}_{= \lambda_{\max}(A)} = L$$

$$\therefore L_{\max} \leq L$$

Complexity

$$\text{gradient descent: } O\left(\frac{L}{m} \log \frac{1}{\epsilon}\right) \cdot O(d^2) = O\left(\frac{L}{m} d^2 \log \frac{1}{\epsilon}\right)$$

$$\text{coordinate descent: } O\left(\frac{d L_{\max}}{m} \log \frac{1}{\epsilon}\right) \cdot O(d) = O\left(\frac{L_{\max}}{m} d^2 \log \frac{1}{\epsilon}\right)$$

Since $L_{\max} \leq L$, coordinate descent has equal or better total complexity than gradient descent.

3 Differentiating the Differentiator

In this problem, we'll derive backpropagation from first principles, connect it to the more general framework of automatic differentiation, and then see how to differentiate through optimization problems using implicit differentiation.

This problem is inspired by [Efficient and Modular Implicit Differentiation](#) (Blondel et al., 2022).

Backpropagation from Scratch

Consider a simple neural network with one hidden layer. The network takes an input $x \in \mathbb{R}^d$, computes a hidden representation $h \in \mathbb{R}^m$, and produces a scalar loss $L \in \mathbb{R}$:

$$z = W_1 x + b_1, \quad h = \sigma(z), \quad \hat{y} = w_2^\top h + b_2, \quad L = \frac{1}{2}(\hat{y} - y)^2,$$

where $W_1 \in \mathbb{R}^{m \times d}$ is a weight matrix, $b_1 \in \mathbb{R}^m$ and $b_2 \in \mathbb{R}$ are biases, $w_2 \in \mathbb{R}^m$ is a weight vector, σ is an elementwise activation function (applied independently to each component of z), and $y \in \mathbb{R}$ is the true label.

Our goal is to compute the gradients of L with respect to every parameter (W_1, b_1, w_2, b_2) so we can update them via gradient descent. The key tool is the **chain rule**: if L depends on a parameter θ only through intermediate quantities $v_1, \dots, v_k$, then

$$\frac{\partial L}{\partial \theta} = \sum_{i=1}^k \frac{\partial L}{\partial v_i} \frac{\partial v_i}{\partial \theta}.$$

The idea of **backpropagation** is to compute these gradients efficiently by working backward from the loss. We first compute how L changes with respect to the output, then propagate that sensitivity back through each layer.

(a) (3 pts) We start at the end and work backward. Find:

- (i) $\frac{\partial L}{\partial \hat{y}}$
- (ii) $\frac{\partial L}{\partial w_2}$ and $\frac{\partial L}{\partial b_2}$
- (iii) $\frac{\partial L}{\partial h_j}$ for each component j , or in vector form, $\frac{\partial L}{\partial h}$.

Note: Observe the pattern: at each step, we multiply the “upstream” derivative $\partial L / \partial \hat{y}$ by a local Jacobian. We never need to re-derive the chain rule from L all the way back — we just pass the upstream signal one layer at a time. This pattern is how we can apply dynamic programming to make this faster.

(b) (3 pts) Now continue propagating backward through the activation function and first layer.

- (i) Show that $\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial h_j} \cdot \sigma'(z_j)$, or in vector form, $\frac{\partial L}{\partial z} = \frac{\partial L}{\partial h} \odot \sigma'(z)$, where $\odot$ denotes elementwise multiplication.
- (ii) Using the result from (i), show that

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z} x^\top \quad \text{and} \quad \frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z}.$$

- (c) (3 pts) Suppose the network has $d = 1000$ input dimensions, $m = 500$ hidden units, and a single scalar output and loss. In your derivations above, you computed all parameter gradients by starting from the scalar $\partial L / \partial \hat{y}$ and propagating backward, performing one matrix–vector product per layer (e.g. multiplying by w_2 , then by elementwise $\sigma'(z)$).

Consider the alternative: for each parameter θ_i , propagate *forward* from θ_i to compute how a small change in θ_i affects L . How many such forward passes would you need? Why is the backward approach (backpropagation) dramatically more efficient when the output is a scalar, as it typically is for a loss function?

a) i. $L = \frac{1}{2} (\hat{y} - y)^2$ $\hat{y} = w_2^T h + b_2$

$$\frac{\partial L}{\partial \hat{y}} = \boxed{\hat{y} - y}$$

ii. $\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2} = \boxed{(\hat{y} - y) \cdot h}$

$$\hookrightarrow \frac{\partial \hat{y}}{\partial w_2} = h$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b_2} = \boxed{(\hat{y} - y)}$$

$$\hookrightarrow \frac{\partial \hat{y}}{\partial b_2} = 1$$

iii. $\frac{\partial L}{\partial h_j} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_j} = \boxed{(\hat{y} - y)(w_2)_j}$

$$\hookrightarrow \frac{\partial \hat{y}}{\partial h_j} = (w_2)_j$$

in vector form,

$$\frac{\partial L}{\partial h} = \boxed{(\hat{y} - y) w_2}$$

b) ii. $\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial h_j} \cdot \frac{\partial h_j}{\partial z_j} = \frac{\partial L}{\partial h_j} \cdot \sigma'(z_j)$ ✓

in vector form,

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial h} \cdot \frac{\partial h}{\partial z} = \frac{\partial L}{\partial h} \odot \sigma'(z)$$
 ✓

ii. $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w_1} = \frac{\partial L}{\partial z} x^T$ ✓

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z} \cdot 1 = \frac{\partial L}{\partial z}$$
 ✓

c) Consider a network w/ $d=1000$ input dims, $m=500$ hidden units, a single scalar output and loss

One forward pass per parameter

$$w_1 = m \times d = 500 \cdot 1000 = 500000$$

$$b_1 = m = 500$$

$$w_2 = m = 500$$

$$b_2 = 1$$

$\boxed{501,001 \text{ forward passes}}$

forward pass requires one pass per parameter, so with each pass costing $O(md)$,

$$\text{total cost} = O(501,000 \cdot md)$$

backward prop computes $\frac{\partial L}{\partial \theta}$ for all parameters in a single backwards pass, so the output is a scalar. You would simply propagate one upstream signal $\frac{\partial L}{\partial \hat{y}}$ backward through each layer with one matrix-vector product each.

$$\text{total cost} = O(md)$$

Automatic Differentiation and Vector-Jacobian Products

Backpropagation is actually a special case of a more general idea: **reverse-mode automatic differentiation** (autodiff). The key concept is the **vector-Jacobian product** (VJP).

Consider a differentiable function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with Jacobian $J_f(x) \in \mathbb{R}^{m \times n}$, where $[J_f(x)]_{ij} = \partial f_i / \partial x_j$. A **vector-Jacobian product** (VJP) computes, for a given vector $v \in \mathbb{R}^m$:

$$v^\top J_f(x) \in \mathbb{R}^{1 \times n}.$$

This multiplies v on the *left* of the Jacobian. If we think of v as an upstream gradient $(\partial L / \partial f)^\top$, then the VJP gives us $(\partial L / \partial x)^\top$ which is exactly what backpropagation computes at each layer.

In contrast, a **Jacobian-vector product** (JVP) computes, for a given vector $u \in \mathbb{R}^n$:

$$J_f(x) u \in \mathbb{R}^m.$$

This multiplies on the *right* and tells us how a perturbation u in the input propagates forward to the output. This corresponds to **forward-mode** autodiff.

Both VJPs and JVPs can be computed without ever forming the full Jacobian matrix $J_f(x)$, which is crucial when n and m are large.

Now consider a computation that is a composition of K functions:

$$L = f_K \circ f_{K-1} \circ \cdots \circ f_1(x),$$

where $f_1 : \mathbb{R}^n \rightarrow \mathbb{R}^{d_1}$, each intermediate $f_k : \mathbb{R}^{d_{k-1}} \rightarrow \mathbb{R}^{d_k}$, and $f_K : \mathbb{R}^{d_{K-1}} \rightarrow \mathbb{R}$ (a scalar loss). By the chain rule, the Jacobian of the full composition is

$$J = J_{f_K} \cdot J_{f_{K-1}} \cdots J_{f_1} \in \mathbb{R}^{1 \times n}.$$

- (d) (8 pts) Consider computing J by chaining together the individual Jacobians. The order of multiplication matters for efficiency.
- (i) Forward mode (right-to-left): We start with a vector $u \in \mathbb{R}^n$ and compute $J_{f_1} u$, then $J_{f_2}(J_{f_1} u)$, and so on. Each step is a JVP. If we want the *full* gradient $J \in \mathbb{R}^{1 \times n}$, we need to repeat this for n different choices of u (the standard basis vectors $e_1, \dots, e_n$). What is the total cost in terms of the number of JVP evaluations?
 - (ii) Reverse mode (left-to-right): We start with the scalar $v = \partial L / \partial L = 1$ and compute $v^\top J_{f_K}$, then $(v^\top J_{f_K}) J_{f_{K-1}}$, and so on. Each step is a VJP. How many passes through the chain are needed to get the full gradient?
 - (iii) In a typical machine learning setting, n (the number of parameters) might be 10^6 or more, while the loss L is a single scalar. Which mode is more efficient, and by what factor? Relate this to your answer in part (c).
- (e) (5 pts) In backpropagation for the one-hidden-layer network from parts (a)–(b), identify the sequence of VJPs that were computed. Specifically, match each step of the backward pass to a VJP $v^\top J_{f_k}$ by identifying what v and f_k are at each stage.

Hint: There are three stages: the loss function, the second layer, and the first layer (including activation). The “upstream” vector v at each stage is the gradient you’ve accumulated so far.

d) ii. We can compute the full gradient J by using each basis vector $(e_1, \dots, e_n)$ to run one forward pass per input dimension,

$$J e_k = \frac{\partial L}{\partial x_k} \quad \leftarrow \text{since each JVP gives one directional derivative}$$

The total cost is n JVP operations.

Propagating all layers! $O(nk)$ JVPs

ii. start with scalar $v=1$ and compute one VJP per layer,

$$1 \cdot J_{f_k} \cdot J_{f_{k-1}} \cdots J_{f_1}$$

each step is one VJP and after k steps, we have the full J , which contains all n partial derivatives simultaneously.

Total cost: $O(k)$ VJP operations.

iii. The reverse mode is more efficient by a factor of n . This result is the same as in part (c) where forward passes corresponds to forward-mode autodiff and backpropagation corresponds to reverse mode autodiff. The efficiency gain hinges on the fact that the output is a scalar $q=1$ which means reverse mode only needs $O(q) = O(1)$ passes regardless of large n . However, if L had n outputs and 1 input, the same logic would apply backwards.

e) $L = f_3 \circ f_2 \circ f_1$

• $f_1(z, h) : \mathbb{R}^d \rightarrow \mathbb{R}^n \Rightarrow$ first layer + activation

• $f_2(h, \hat{y}) : \mathbb{R}^n \rightarrow \mathbb{R} \Rightarrow$ second layer

• $f_3(\hat{y}, L) : \mathbb{R} \rightarrow \mathbb{R} \Rightarrow$ loss

loss function: $f_k = L(\hat{y}) = \frac{1}{2}(\hat{y} - y)^2$

$$v = 1 = \frac{\partial L}{\partial L}$$

$$J_{f_k} = \frac{\partial L}{\partial \hat{y}} = (\hat{y} - y)$$

$$\text{VJP: } v^T J_{f_k} = 1 \cdot (\hat{y} - y) = (\hat{y} - y)$$

2nd layer: $f_k = \hat{y}(h) = w_2^T h + b_2$

$$v = (\hat{y} - y)$$

$$J_{f_k} = \frac{\partial \hat{y}}{\partial h} = w_2$$

$$\text{VJP} = v^T J_{f_k} = (\hat{y} - y) w_2$$

1st layer: $f_k = h(z) = \sigma(w_1 x + b_1)$

$$v = (\hat{y} - y) w_2$$

$$J_{f_k} = \frac{\partial h}{\partial z} = \text{diag}(\sigma'(z)) \cdot w_1$$

$$\text{VJP} = v^T J_{f_k} = (\hat{y} - y) w_2 \cdot \text{diag}(\sigma'(z)) \cdot w_1$$

$$\frac{\partial L}{\partial x} = w_1^T (\sigma'(z) \odot (\hat{y} - y) w_2)$$

Implicit Differentiation

The autodiff framework above handles any computation expressible as a composition of elementary operations. However, some computations are defined *implicitly* — for instance, as the solution to an optimization problem — and cannot be written as an explicit formula to differentiate through. Implicit differentiation extends gradient computation to these settings.

Automatic differentiation handles any computation built from elementary operations. But what if part of your computation involves *solving an optimization problem*?

Suppose we have an objective $f(x, \theta)$ and define

$$x^*(\theta) = \operatorname{argmin}_{x \in \mathbb{R}^d} f(x, \theta),$$

where $\theta \in \mathbb{R}^n$ are parameters we want to differentiate through. There is no explicit formula for $x^*(\theta)$ since it's defined implicitly as the solution to an optimization problem. So we can't just apply autodiff to a formula.

However, if f is differentiable and $x^*(\theta)$ is an unconstrained minimum, we know the gradient must vanish:

$$F(x^*(\theta), \theta) := \nabla_x f(x^*(\theta), \theta) = 0.$$

This is the optimality condition, and it implicitly relates x^* to θ . The **implicit function theorem** tells us that if F is continuously differentiable and the Jacobian $\partial_1 F(x^*, \theta)$ (i.e., the Hessian $\nabla_x^2 f$) is invertible, then $x^*(\theta)$ is a differentiable function of θ in a neighborhood of the current value.

To find the Jacobian $\partial x^*/\partial \theta$, we differentiate the optimality condition $F(x^*(\theta), \theta) = 0$ with respect to θ using the chain rule:

$$\underbrace{\partial_1 F(x^*, \theta)}_{A \in \mathbb{R}^{d \times d}} \underbrace{\frac{\partial x^*}{\partial \theta}}_{J \in \mathbb{R}^{d \times n}} + \underbrace{\partial_2 F(x^*, \theta)}_{B \in \mathbb{R}^{d \times n}} = 0,$$

which gives $AJ = -B$, or equivalently $J = -A^{-1}B$. For the stationary point condition $F = \nabla_x f$, this becomes

$$\frac{\partial x^*}{\partial \theta} = - \left[\nabla_x^2 f(x^*, \theta) \right]^{-1} \partial_\theta \nabla_x f(x^*, \theta).$$

(f) (8 pts) **Implicit diff in action.** Consider the ridge regression problem:

$$x^*(\lambda) = \operatorname{argmin}_{x \in \mathbb{R}^d} \frac{1}{2} \|Ax - b\|^2 + \frac{\lambda}{2} \|x\|^2,$$

where $A \in \mathbb{R}^{m \times d}$, $b \in \mathbb{R}^m$, and $\lambda > 0$.

- (i) Write down the optimality condition $F(x, \lambda) = 0$ (i.e., set the gradient with respect to x equal to zero).
- (ii) Compute the Jacobian $\partial_1 F$ (with respect to x) and $\partial_2 F$ (with respect to λ). Use the implicit differentiation formula to derive $\partial x^*/\partial \lambda$.
- (iii) Verify your answer by differentiating the closed-form solution $x^*(\lambda) = (A^\top A + \lambda I)^{-1} A^\top b$ directly.

Hint: Use the identity $\frac{d}{d\lambda} M(\lambda)^{-1} = -M(\lambda)^{-1} \frac{dM}{d\lambda} M(\lambda)^{-1}$ for an invertible matrix $M(\lambda)$.

f) i. $F(x, \lambda) = 0$

$$\nabla_x f(x, \lambda) = A^T(Ax - b) + \lambda x = \boxed{(A^T A + \lambda I)x - A^T b = 0}$$

ii. $\frac{\partial F}{\partial x} = A^T A + \lambda I$

$$\frac{\partial F}{\partial \lambda} = x$$

$$\frac{\partial x^*}{\partial \lambda} = - \left[\nabla_x^2 f(x, \lambda) \right]^{-1} \nabla_\lambda \nabla_x f(x^*, \lambda) = -(\partial_1 F)^{-1} \partial_2 F$$

$$= \boxed{-(A^T A + \lambda I)^{-1} x^*(\lambda)}$$

iii. $\frac{\partial x^*}{\partial \lambda} = \frac{d}{d\lambda} (A^T A + \lambda I)^{-1} A^T b = - (A^T A + \lambda I)^{-1} \cdot I \cdot \underbrace{(A^T A + \lambda I)^{-1} A^T b}_{x^*(\lambda)}$

$\hookrightarrow$ apply $\frac{d}{d\lambda} M(\lambda)^{-1} = -M(\lambda)^{-1} \frac{dM}{d\lambda} M(\lambda)^{-1}$ $\rightarrow x^*(\lambda)$

$$= \boxed{-(A^T A + \lambda I)^{-1} x^*(\lambda)} \quad \checkmark$$

(g) (8 pts) **Why not just unroll?** An alternative to implicit differentiation is **unrolling**: run T steps of gradient descent to approximate $x^*(\theta)$, and then differentiate through all T steps using standard autodiff.

- (i) In reverse-mode autodiff (backpropagation), we must store the intermediate values at each step for the backward pass. If each gradient descent iterate $x_t \in \mathbb{R}^d$, what is the memory cost of differentiating through T unrolled steps as a function of d and T ?
- (ii) Implicit differentiation only requires the final solution x^* (or an approximation $\hat{x}$) and the ability to solve a single linear system $AJ = -B$. What is the memory cost in terms of d ? Does it depend on T ?
- (iii) In the ridge regression example from (f), suppose $d = 10,000$ and convergence requires $T = 1,000$ gradient steps. Briefly compare the memory and computation tradeoffs of unrolling vs. implicit differentiation for this problem.

Note: For ridge regression, $A = A^\top A + \lambda I \in \mathbb{R}^{d \times d}$. Forming this matrix from the $m \times d$ data matrix costs $O(md)$, not $O(d^2)$. Storing it explicitly still costs $O(d^2)$, but if an iterative linear solver is used, one only needs $O(d)$ memory for x^* .

(h) (8 pts) **When does implicit differentiation break down?**

The implicit function theorem requires that $A = \partial_1 F(x^*, \theta)$ be invertible. For the unconstrained problem $x^*(\theta) = \operatorname{argmin}_x f(x, \theta)$, we have $A = \nabla_x^2 f(x^*, \theta)$, the Hessian.

- (i) Give a simple concrete example of a function $f(x, \theta)$ (with $x, \theta \in \mathbb{R}$) where the Hessian at the minimizer is zero, and implicit differentiation fails. What goes wrong geometrically at this point?

Hint: Think of a function whose minimum is “very flat.”

- (ii) In practice, solvers don’t find $x^*(\theta)$ exactly — they return an approximation $\hat{x}$. The paper by Blondel et al. shows that the error in the Jacobian estimate satisfies

$$\left\| \hat{J} - \frac{\partial x^*}{\partial \theta} \right\| \leq C \|\hat{x} - x^*(\theta)\|$$

for a constant C that depends on the conditioning of A . Explain intuitively why a poorly conditioned A (eigenvalues close to zero) would make this bound worse, connecting back to your example from (i).

g) i. Backprop requires storing all intermediate iterates per pass...

$$\text{Memory} = O(Td)$$

with T steps and d input dimensions

ii. Since implicit differentiation only requires x^* to be stored, plus solving $AJ = -B$, which requires matrices $\in \mathbb{R}^{d \times d}$,

$$\text{Memory} = O(d^2)$$

This does not depend on T

iii. $d=10,000$ $T=1,000$

Memory

$$\text{Unrolling: } O(Td) = O(10^7)$$

$$\text{Implicit diff: } O(d^2) = O(10^8)$$

Computation

Unrolling: $O(Tmd)$ is T GD steps, no solve

Implicit diff: $O(md+d^2)$ -- run solver once, then one linear solve

Unrolling is cheaper in memory if $T < d$, but implicit diff is faster at computing.

h) i. Example where Hessian is zero:

$$f(x, \theta) = \theta x^4$$

minimizer $x^*(\theta) = 0$ for all θ

$$\nabla_x^2 f(x, \theta) = 12\theta x^2|_{x=0} = 0$$

$A = 0$ which is not invertible, so implicit diff fails.

Geometrically, the function is ultra-flat at $x^*=0$ where it has a 4th-order tangency with the x -axis. The curvature, or 2nd derivative is zero, meaning the optimizer cannot distinguish which direction x^* moves based on θ .

ii. implicit diff formula;

$$\frac{\partial x^*}{\partial \theta} = -A^{-1}B = -[\nabla_x^2 f]^{-1} \partial_\theta \nabla_x f$$

When A is poorly conditioned, or its smallest eigenvalue is close to zero, then A^{-1} has very large eigenvalues. The error bound is

$$\|\hat{J} - \frac{\partial x^*}{\partial \theta}\| \leq C \|\hat{x} - x^*(\theta)\|$$

where $C \propto \|A^{-1}\| = \frac{1}{\lambda_{\min}(A)}$, so small errors in $\hat{x}$ get amplified by $\frac{1}{\lambda_{\min}(A)}$.

So, as $\lambda_{\min}(A) \rightarrow 0$, $C \rightarrow \infty$ and the gradient becomes flat at the minimum. This means that mapping $\theta \mapsto x^*(\theta)$ is nearly non-differentiable, so any estimate in x^* is highly inaccurate.

4 I Can't Believe It's Not Distortion!

In lecture, we've briefly covered information theory and entropy in the context of classification. Now, we'll review what we've learned and apply it to learning compressions and neural networks.

This problem is inspired by [The information bottleneck method](#) and [Opening the black box of Deep Neural Networks via Information](#).

Information Review

Here's a brief review of information theory as described in lecture (with a little more!). We use the convention of $\log$ being $\log_2$.

The **Shannon information content** of an outcome x is defined to be

$$h(x) = \log \frac{1}{p(x)}$$

measured in bits. The **entropy** of an ensemble X is defined as the average Shannon information content of an outcome:

$$H(X) = \sum_{x \in \mathcal{A}_X} p(x) \log \frac{1}{p(x)}$$

where $\mathcal{A}_X$ (sometimes called the alphabet) contains the set of all possible values the random variable X can take. It can also be written as $H(p)$, where p is the density function of X . Note some properties of entropy:

1. $H(X) \geq 0$ with equality iff $p(x) = 1$ for one $x \in \mathcal{A}_X$.
2. Entropy is maximized if $p(X)$ is uniform.

The **relative entropy** or **Kullback-Leibler divergence** between two densities $p(X)$ and $q(X)$ defined over the same alphabet $\mathcal{A}_X$ is

$$D_{\text{KL}}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

Note that the relative entropy is not symmetric in the exchange of p and q .

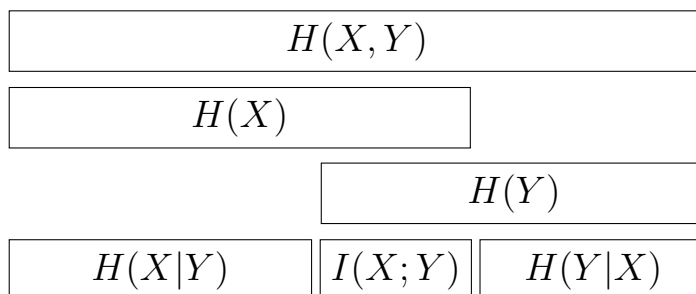
The **mutual information** between X and Y is

$$I(X; Y) = H(X) - H(X|Y)$$

and satisfies $I(X; Y) = I(Y; X)$, $I(X; Y) \geq 0$. Similarly, the **conditional mutual information** between X and Y given Z is

$$I(X; Y|Z) = H(X|Z) - H(X|Y, Z)$$

Take a look at the following diagram for the relationship between joint entropy $H(X, Y)$, marginal entropies $H(X)$ and $H(Y)$, conditional entropy $H(X|Y)$, $H(Y|X)$, and mutual information $I(X; Y)$.



For additional review on information theory and convexity, please read through Section 2.5 of the Bishop textbook.

Surprise!

- (a) (3 pts) Show that relative entropy satisfies the Gibbs' inequality $D_{\text{KL}}(p||q) \geq 0$, with equality only if $p = q$.

Hint: Try using Jensen's inequality $\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$.

Hint: Remember that p and q are probability distributions over X .

- (b) (5 pts) Assume X is a discrete random variable over alphabet $\mathcal{A}_X$ with $|\mathcal{A}_X| = m$ and distribution p . Let q be the uniform distribution over $\mathcal{A}_X$.

- (i) Apply Gibbs' inequality to $D_{\text{KL}}(p||q)$ to show $H(X) \leq \log m$, with equality iff p is uniform.
- (ii) Assume we design a binary code for X as seen in lecture, assigning codeword lengths $\ell(x)$ to each outcome. From the Kraft inequality, we know that any uniquely decodable code must satisfy

$$\sum_{x \in \mathcal{A}_X} 2^{-\ell(x)} \leq 1$$

Define the distribution

$$q(x) = \frac{2^{-\ell(x)}}{C}$$

where $C = \sum_{x'} 2^{-\ell(x')}$. Apply Gibbs' inequality to $D_{\text{KL}}(p||q)$ to show that the expected codeword length satisfies

$$\mathbb{E}_p[\ell(X)] \geq H(X)$$

a) Relative entropy:

$$D_{KL}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = - \sum_x p(x) \log \frac{q(x)}{p(x)}$$

Jensen's ineq: $E[\log Z] \leq \log E[Z]$

$$\Rightarrow -E[\log Z] \geq -\log E[Z]$$

$$D_{KL}(p \parallel q) = - \sum_x p(x) \log \frac{q(x)}{p(x)} = -E\left[\log \frac{q(x)}{p(x)}\right] \geq -\log\left(E\left[\frac{q(x)}{p(x)}\right]\right) = -\log(1) = 0$$

$$\hookrightarrow E\left[\frac{q(x)}{p(x)}\right] = \sum_x p(x) \cdot \frac{q(x)}{p(x)} = \sum_x q(x) = 1$$

$$\therefore D_{KL}(p \parallel q) \geq 0$$

Jensen's inequality only holds with equality iff the argument is constant, so

$$\frac{q(x)}{p(x)} = c$$

But, $q, p = 1$ since they are densities, so $c = 1$, or $p(x) = q(x)$ for all x . Thus,

$$D_{KL}(p \parallel q) = 0 \Leftrightarrow p = q$$

$$b) i. D_{KL}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = \sum_x p(x) \log \frac{p(x)}{1/n} = \sum_x p(x) [\log p(x) + \log n] = -H(X) + \log n \geq 0$$

$$\Rightarrow H(X) \leq \log n \quad \checkmark$$

$$\hookrightarrow -\left(\sum_x p(x) \log \frac{1}{p(x)}\right) = \sum_x p(x) \log p(x) \\ -\log\left(\frac{1}{n}\right) = \log n$$

$$ii. q(x) = \frac{2^{-R(x)}}{C}$$

$$D_{KL}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = \sum_x p(x) \log \frac{p(x) \cdot C}{2^{-R(x)}} \geq 0 \Rightarrow \sum_x p(x) [\log p(x) + R(x) + \log C] \geq 0$$

$$\Rightarrow -H(X) + E_p[R(X)] + \log C \geq 0 \Rightarrow E_p[R(X)] \geq H(X) - \log C$$

Using Kraft inequality, $C = \sum_x 2^{-R(x)} \leq 1$, so $\log C \leq 0$

$$E_p[R(X)] \geq H(X) - \log C \geq H(X)$$

$$\Rightarrow \boxed{E_p[R(X)] \geq H(X)}$$

The Right Measure of Distortion

In lossy compression, we accept some loss of information when compressing a signal X into a shorter representation $\tilde{X}$. The quality of a compression is measured by a **distortion function** $d(x, \tilde{x})$, which quantifies how “far” the compressed version $\tilde{x}$ is from the original x . For example, in image compression we might use mean squared error between pixel values.

Rate distortion theory characterizes the fundamental tradeoff: to achieve an average distortion of at most D , we need at least $R(D)$ bits per symbol, where

$$R(D) = \min_{\{p(\tilde{x}|x): \langle d(x, \tilde{x}) \rangle \leq D\}} I(X; \tilde{X}).$$

The catch is that we must *choose* the distortion function d in advance, and this choice implicitly determines which features of X matter. In practice, we rarely know the right distortion function.

The **information bottleneck** (IB) as described in [Tishby et al.](#) sidesteps this problem. Instead of specifying a distortion function, we specify a **relevance variable** Y that tells us what matters. We then compress X into $\tilde{X}$ while preserving as much information about Y as possible. Because $\tilde{X}$ is computed from X alone (without looking at Y directly), the three variables form a Markov chain:

$$Y \longleftrightarrow X \longrightarrow \tilde{X}$$

Here, $\longleftrightarrow$ denotes an arbitrary (undirected) statistical dependence between Y and X (i.e., they may be correlated in any way), while $\longrightarrow$ denotes a directed dependency ($\tilde{X}$ is a deterministic function of X). This means Y and $\tilde{X}$ are conditionally independent given X : all the information that $\tilde{X}$ has about Y flows through X . A consequence of this structure is the **data processing inequality**: $I(\tilde{X}; Y) \leq I(X; Y)$. Compression can only lose information about Y , never gain it.

The IB objective balances compression against prediction by minimizing

$$\mathcal{L}[p(\tilde{x}|x)] = I(\tilde{X}; X) - \beta I(\tilde{X}; Y),$$

where $\beta > 0$ controls the tradeoff. When β is small we compress aggressively (small $I(\tilde{X}; X)$); when β is large we prioritize preserving information about Y .

Now, consider a hospital that compresses patient test results $X \in \{0, 1, 2, 3\}$ into coarser diagnostic categories $\tilde{X} \in \{a, b\}$ to save storage. The variable $Y \in \{0, 1\}$ indicates disease status. The joint distribution is given by

x	$p(x)$	$p(y = 1 x)$
0	0.4	0.05
1	0.2	0.15
2	0.3	0.80
3	0.1	0.95

(c) (2 pts) Consider two candidate partitions of $\{0, 1, 2, 3\}$ into two non-empty groups:

- **Partition A:** $\{0, 1\} \mid \{2, 3\}$ (group by disease risk: low vs. high)
- **Partition B:** $\{0, 2\} \mid \{1, 3\}$ (group by parity of the test result)

For each partition, compute the induced conditional distribution $p(y | \tilde{x})$ for each group using

$$p(y | \tilde{x} = a) = \frac{\sum_{x \in \text{group } a} p(x) p(y | x)}{\sum_{x \in \text{group } a} p(x)}.$$

Then compute $I(\tilde{X}; Y)$ for both partitions. Which partition retains more information about Y , and why does this make intuitive sense in terms of what each partition “pays attention to”?

- (d) (15 pts) Now we will see that the information bottleneck automatically discovers the right notion of distortion — without anyone specifying a distortion function in advance.

- (i) Prove that for any deterministic compression $X \rightarrow \tilde{X}$,

$$I(X; Y) - I(\tilde{X}; Y) = \mathbb{E}_p \left[D_{\text{KL}}(p(y|X) \| p(y|\tilde{X})) \right] = \sum_x p(x) D_{\text{KL}}(p(y|x) \| p(y|\tilde{x}(x))),$$

where $\tilde{x}(x)$ denotes the codeword assigned to x .

Hint: Start by writing $I(X; Y) - I(\tilde{X}; Y) = H(Y|\tilde{X}) - H(Y|X)$. Then expand $H(Y|\tilde{X})$ by replacing $p(\tilde{x}) p(y|\tilde{x})$ with $\sum_{x \in \tilde{x}} p(x) p(y|x)$ and compare term by term with $H(Y|X)$.

- (ii) **Numerical verification.** Using **Partition A** ($\{0, 1\} | \{2, 3\}$) from part (c), numerically verify the identity from (i):

$$I(X; Y) - I(\tilde{X}; Y) = \sum_x p(x) D_{\text{KL}}(p(y|x) \| p(y|\tilde{x}(x))).$$

Compute the left-hand side using your $I(X; Y)$ and I_A from part (c), and compute the right-hand side directly by evaluating the per- x KL divergences. (Answers up to two decimal places are fine.)

- (iii) Using the identity you just proved: *without doing any additional computation*, which of Partition A or Partition B from part (c) has a smaller expected KL divergence $\mathbb{E}_p[D_{\text{KL}}(p(y|X) \| p(y|\tilde{X}))]$? Explain your reasoning in one sentence.
- (iv) The identity in (i) tells us that maximizing $I(\tilde{X}; Y)$ is equivalent to minimizing

$$\mathbb{E}_p[D_{\text{KL}}(p(y|X) \| p(y|\tilde{X}))]$$

In classical rate-distortion theory, one must *choose* a distortion function $d(x, \tilde{x})$ in advance. Explain in 2–3 sentences what the information bottleneck framework reveals about distortion: what plays the role of the distortion function, and why is it notable that nobody had to choose it?

c) Partition A: $\{0,1\} | \{2,3\}$

Group a: $\{0,1\}$, $p(\tilde{x}=a) = 0.4 + 0.2 = 0.6$

$$p(y=1 | \tilde{x}=a) = \frac{0.4(0.05) + 0.2(0.15)}{0.6} = \frac{0.02 + 0.03}{0.6} = \frac{0.05}{0.6} \approx 0.0833$$

Group b: $\{2,3\}$, $p(\tilde{x}=b) = 0.3 + 0.1 = 0.4$

$$p(y=1 | \tilde{x}=b) = \frac{0.3(0.8) + 0.1(0.95)}{0.4} = \frac{0.335}{0.4} = 0.8375$$

$$I(\tilde{X}; Y) = H(Y) - H(Y | \tilde{X}) = 0.961 - 0.490 = \boxed{0.471 \text{ bits}}$$

$$\hookrightarrow H(Y) = -0.385 \log(0.385) - (1-0.385) \log(1-0.385) \approx 0.961 \text{ bits}$$

$$H(Y | \tilde{x}=a) = -0.8833 \log(0.0833) - (1-0.8833) \log(1-0.8833) \approx 0.392 \text{ bits}$$

$$H(Y | \tilde{x}=b) = -0.8375 \log(0.8375) - (1-0.8375) \log(1-0.8375) \approx 0.637 \text{ bits}$$

$$H(Y | \tilde{X}) = p(\tilde{x}=a) H(Y | \tilde{x}=a) + p(\tilde{x}=b) H(Y | \tilde{x}=b) = 0.6(0.392) + 0.4(0.637) = 0.490 \text{ bits}$$

Partition B: $\{0,2\} | \{1,3\}$

Group a: $\{0,2\}$, $p(\tilde{x}=a) = 0.4 + 0.3 = 0.7$

$$p(y=1 | \tilde{x}=a) = \frac{0.4(0.05) + 0.3(0.80)}{0.7} = \frac{0.26}{0.7} \approx 0.371$$

Group b: $\{1,3\}$, $p(\tilde{x}=b) = 0.2 + 0.1 = 0.3$

$$p(y=1 | \tilde{x}=b) = \frac{0.2(0.15) + 0.1(0.95)}{0.3} = \frac{0.125}{0.3} \approx 0.417$$

$$I(\tilde{X}; Y) = H(Y) - H(Y | \tilde{X}) = 0.961 - 0.964 \approx -0.003 \text{ bits} \approx \boxed{0 \text{ bits}}$$

$$\hookrightarrow H(Y) = 0.961 \text{ (part A)}$$

$$H(Y | \tilde{x}=a) = -0.371 \log(0.371) - (1-0.371) \log(1-0.371) \approx 0.959 \text{ bits}$$

$$H(Y | \tilde{x}=b) = -0.417 \log(0.417) - (1-0.417) \log(1-0.417) \approx 0.986 \text{ bits}$$

$$H(Y | \tilde{X}) = 0.7(0.959) + 0.3(0.986) = 0.964 \text{ bits}$$

Partition A holds far more information than B about Y (≈ 0.471 bits) which is likely due to A grouping by disease risk. $\{0,1\}$ are low risk while $\{2,3\}$ are high risk. Partition B groups in parity which is unrelated to disease status, so $Y | \tilde{X}$ is similar to just Y.

d) i. Prove $I(X; Y) - I(\tilde{X}; Y) = E_p[D_{KL}(p(y|x) || p(y|\tilde{x}(x)))]$

$$\text{Use } I(X; Y) - I(\tilde{X}; Y) = H(Y | \tilde{X}) - H(Y | X)$$

$$\hookrightarrow H(Y | \tilde{X}) = - \sum_{\tilde{x}} p(\tilde{x}) \sum_Y p(y | \tilde{x}) \log p(y | \tilde{x}) = - \sum_{\tilde{x}} \sum_{x \in \tilde{x}} p(x) \sum_Y p(y | x) \log p(y | \tilde{x}(x))$$

$$H(Y | X) = - \sum_x p(x) \sum_Y p(y | x) \log p(y | x)$$

$$p(\tilde{x}) p(y | x) = \sum_{x \in \tilde{x}} p(x) p(y | x)$$

$$\Rightarrow \sum_x p(x) \sum_Y p(y | x) \log p(y | x) - \sum_{\tilde{x}} \sum_{x \in \tilde{x}} p(x) \sum_Y p(y | x) \log p(y | \tilde{x}(x)) = \sum_x p(x) \sum_Y p(y | x) [\log p(y | x) - \log p(y | \tilde{x}(x))]$$

$$= \sum_x p(x) \sum_Y p(y | x) \log \frac{p(y | x)}{p(y | \tilde{x}(x))} = \sum_x p(x) D_{KL}(p(y | x) || p(y | \tilde{x}(x))) = E_p[D_{KL}(p(y | x) || p(y | \tilde{x}(x)))] \quad \checkmark$$

ii. LHS: $I(X;Y) - I_A \approx I(X;Y) - 0.471 = 0.479 - 0.471 = 0.008$ bits

$\hookrightarrow I(X;Y) = H(Y) - H(Y|X) = 0.961 - 0.482 = 0.479$ bits

$\hookrightarrow H(Y|X) = \sum_x p(x) H(Y|X=x) = 0.4 H(0.05) + 0.2 H(0.15) + 0.3 H(0.8) + 0.1 H(0.95) \approx 0.482$

RHS: $\sum_x D_{KL}(p(y|x) \| p(y|\hat{x}(x))) \approx 0.008$

group a = {0,1}, $p(y=1|\hat{x}=a) \approx 0.0833$

$x=0$: $D_{KL} = 0.05 \log \frac{0.05}{0.0833} + 0.95 \log \frac{0.95}{1-0.0833} \approx 0.018$ bits

$x=1$: $D_{KL} = 0.15 \log \frac{0.15}{0.0833} + 0.85 \log \frac{0.85}{1-0.0833} \approx 0.008$ bits

group b = {2,3}, $p(y=1|\hat{x}=b) = 0.8375$

$x=2,3$: D_{KL} near 0 since $p(y=1|x=2) = 0.80 \approx 0.8375$ and $p(y=1|x=3) = 0.95$

iii. By the identity $I(X;Y) - I(\hat{X};Y) = E[D_{KL}]$ and since $I_A > I_B$, we have

$I(X;Y) - I_A < I(X;Y) - I_B$, so Partition A has the smaller expected KL divergence since it loses less information about Y , meaning each $p(y|x)$ is closer to its group's $p(y|\hat{x})$

iv. This identity shows that maximizing $I(\hat{X};Y)$ is equivalent to minimizing $E[D_{KL}(p(y|x) \| p(y|\hat{x}(x)))]$, which is the average KL divergence between a point's true label distribution and its codeword's label distribution. This is the foundation for the distortion function, where IB reveals that the correct way to measure compression error is by how much the compressed representation distorts the conditional distribution over Y . Nobody chose this as it was implied by the relevance variable Y .

Okay, But Can We Get to Neural Networks Already?

A deep neural network processes an input X through successive hidden layers $T_1, T_2, \dots, T_K$ to produce a prediction $\hat{Y}$ of a target variable Y . Because each layer is computed only from the previous one, the variables form a **Markov chain**:

$$Y \longleftrightarrow X \longrightarrow T_1 \longrightarrow T_2 \longrightarrow \dots \longrightarrow T_K \longrightarrow \hat{Y}.$$

(Recall: $\leftrightarrow$ denotes an arbitrary undirected dependence, while $\rightarrow$ denotes a directed functional dependency.) This means that all information any layer T_i has about Y must flow through X , and all information T_{i+1} has about anything must flow through T_i .

We can characterize what each layer “knows” by two numbers: $I(X; T_i)$, measuring how much detail about the input it retains, and $I(T_i; Y)$, measuring how much it knows about the target. Plotting each layer as a point $(I(X; T_i), I(T_i; Y))$ gives the **information plane**.

Note: mutual information is invariant under invertible transformations: if ϕ is an invertible function, then $I(X; \phi(Y)) = I(X; Y)$.

- (e) (3 pts) Consider a single hidden layer T computed from X , forming the chain $Y \leftrightarrow X \rightarrow T$. Using the data processing inequality, we can show that $I(T; Y) \leq I(X; Y)$.

If we impose a further constraint that $T = f(X)$ where f is a deterministic invertible function (e.g. a simple multiplication of an invertible matrix), we can now show $I(T; Y) = I(X; Y)$, and $I(T; X) = H(X)$ with the invariance of mutual information under invertible maps (this is left as an optional exercise).

Such a hidden layer T cannot compress the input (why?). For a layer to actually compress X (reduce $I(X; T)$ below $H(X)$), explain in one sentence what property the mapping $X \rightarrow T$ must have, and what component of a typical layer is responsible for adding this property.

- (f) (5 pts) Consider a network with two hidden layers forming the chain $Y \leftrightarrow X \rightarrow T_1 \rightarrow T_2 \rightarrow \hat{Y}$.

- (i) We say T is a **sufficient statistic** of X for Y if $I(T; Y) = I(X; Y)$ —that is, T captures everything X knows about Y . Suppose T_1 is a sufficient statistic of X for Y . Must T_2 also be a sufficient statistic? Either prove it or give a concrete counterexample.

Hint: Try setting up X as a random variable with an alphabet with Y as a function of $X = x$. Then, with $T_1 = X$ (the identity), as an exercise, try finding a function $T_2 = g(T_1)$ that destroys information about Y .

- (ii) Explain in a few sentences why adding more layers to a neural network is *not* about increasing the information available about Y —each layer can only lose information. What, then, might be the benefit of depth? (This question is open-ended; a thoughtful hypothesis is sufficient.)

- (g) (10 pts) The paper mentioned earlier, [Schwartz-Ziv and Tishby \(2017\)](#), found that training a deep neural network with stochastic gradient descent (SGD) has two distinct phases, visible in the information plane:

Phase 1 (fitting): All layers rapidly increase $I(T_i; Y)$ — they learn to predict the labels. Gradient updates are large and consistent (high signal-to-noise ratio).

Phase 2 (compression): The layers slowly decrease $I(X; T_i)$ — they discard irrelevant input details. Gradient updates become noisy with small means (low signal-to-noise ratio), so the weights evolve approximately as random walks.

Let's explore *why* adding noise to a representation can compress it, and why this compression can be beneficial.

- (i) Let $T \in \{0, 1\}$ be a binary hidden representation with $P(T = 1) = \pi$, and let T' be a noisy copy of T : independently with probability ε , T' flips the value of T (and with probability $1 - \varepsilon$ it keeps it). Show that

$$I(X; T') = H(T') - H(\varepsilon),$$

where $H(\varepsilon) = -\varepsilon \log \varepsilon - (1 - \varepsilon) \log(1 - \varepsilon)$ is the binary entropy of the noise.

Hint: Remember that constructing a Markov chain representation can be beneficial!

- (ii) Using your result from (i): as ε increases from 0 to $1/2$, what happens to $I(X; T')$? What is $I(X; T')$ when $\varepsilon = 1/2$, and why does this make sense?
- (iii) Now here is the key question. Adding noise reduces $I(X; T')$ (compression), but it also risks reducing $I(T'; Y)$ (losing relevant information). Explain why the *irrelevant* information in a neural network's representation is more “fragile” (easily destroyed by noise) than the *relevant* information.

Hint. Think about what kinds of input distinctions are encoded in fine-grained weight values (that small noise can disrupt) versus coarse structural features (that are robust to perturbation). As a concrete example: if Y is a binary label and T encodes which side of the decision boundary X falls on, versus the exact distance from the boundary, which is more easily disrupted by small weight noise?

e) If $T=f(X)$ with invertible f , then $X=f^{-1}(X)$ is recoverable from T , so $I(T;X) = H(X)$, meaning no information about X is lost. To actually compress T , the mapping $X \rightarrow T$ must be many-to-one, or injective, and it is the nonlinear activation function such as ReLU or sigmoid that introduces this property by collapsing distinct input values to the same output.

f) i. No, T_2 is always a sufficient statistic if T_1 is.

Counterexample: Let $X \in \{0,1,2,3\}$ with $Y = 1[X \in \{2,3\}]$, set $T_1 = X$, $I(T_1;Y) = I(X;Y)$, so T_1 is sufficient. Now let $T_2 = g(T_1) = X \bmod 2$ (parity)

$$T_2 = 0 \text{ when } X \in \{0,2\}, T_2 = 1 \text{ when } X \in \{1,3\}$$

If $p(X)$ is uniform, then knowing T_2 tells us parity, but not whether $X \geq 2$, so $I(T_2;Y) < I(X;Y)$ - T_2 is not a sufficient statistic given!

$$I(T_2;Y) \leq I(T_1;Y) = I(X;Y)$$

ii. By the data processing inequality, each layer can only lose information about Y , so depth cannot increase $I(T_i;Y)$. The benefit is likely about representational efficiency and compositionality, where deeper networks can represent the same function with exponentially fewer parameters than shallow ones. So, in that sense, a good MN compresses away irrelevant information about X , while preserving $I(T_i;Y)$ near $I(X;Y)$.

g) i. Prove $I(X;T') = H(T') - H(\epsilon)$

MC: $X \rightarrow T \rightarrow T'$ where T' is a noisy copy of T

Use identity: $I(X;T') = H(T') - H(T'|X)$

$$\hookrightarrow H(T'|X) = H(T'|T) = H(\epsilon)$$

T' flips w/ probability ϵ

$$H(\epsilon) = -\epsilon \log \epsilon - (1-\epsilon) \log (1-\epsilon)$$

$$\therefore I(X;T') = H(T') - H(\epsilon)$$

ii. $\epsilon: 0 \rightarrow \frac{1}{2}$

From above,

- $H(\epsilon)$ increases as ϵ increases, maximized at $\frac{1}{2}$

- $H(T')$ tends to 1 bit as ϵ increases as $T' \rightarrow \text{Bernoulli}(\frac{1}{2})$

at $\epsilon = \frac{1}{2}$, $H(\epsilon) = H(\frac{1}{2}) = 1$ bit and $T' \sim \text{Bernoulli}(\frac{1}{2})$

$$\therefore I(X;T')_{\epsilon=\frac{1}{2}} = 1 - 1 = \underline{0 \text{ bits}}$$

iii. Relevant information about Y is quite coarse as it categorizes inputs on a decision boundary that is stable under small perturbations.

Irrelevant information is, by contrast, quite fine as it computes exact distance from the boundary and are easily disturbed by small weight noise.

The noise acts as a selective filter which destroys the fragile fine-grained details while leaving coarse structure containing information about Y .